

The Tinkerbell Chaotic Map

1. Introduction to Chaotic Maps

What are Chaotic Maps?

Chaotic maps are discrete-time dynamical systems that exhibit chaotic behavior. They are characterized by:

- **Sensitivity to initial conditions** (the butterfly effect)
- **Topological mixing**
- **Dense periodic orbits**
- **Deterministic yet unpredictable behavior**

Despite their simple mathematical formulations, chaotic maps can produce incredibly complex and seemingly random behavior, making them valuable for various applications including cryptography.

The Tinkerbell Map

The Tinkerbell map is a two-dimensional discrete-time dynamical system defined by the following equations:

$$x_{n+1} = x_n^2 - y_n^2 + a \cdot x_n + b \cdot y_n$$

$$y_{n+1} = 2 \cdot x_n \cdot y_n + c \cdot x_n + d \cdot y_n$$

Where:

- (x_n, y_n) represents the state at iteration n
- a , b , c , and d are system parameters that control the behavior

The name "Tinkerbell" comes from the fairy-like patterns that emerge when plotting the system's trajectory, reminiscent of Tinker Bell's flight path in Disney films.

2. Mathematical Properties of the Tinkerbell Map

Parameter Spaces

The behavior of the Tinkerbell map is highly dependent on its parameters. Common parameter values include:

- $a = 0.9$, $b = -0.6013$, $c = 2.0$, $d = 0.5$ (classic parameters)
- $a = 0.3$, $b = 0.6000$, $c = 2.0$, $d = 0.27$ (alternative parameters)

In our cryptographic implementation, we use parameters within these ranges:

- $a \in [-1.2, -0.3]$
- $b \in [-1.0, -0.2]$
- $c \in [1.5, 2.5]$
- $d \in [0.3, 1.3]$

Chaotic Behavior

The Tinkerbell map exhibits several key properties that make it valuable for cryptography:

1. **Sensitivity to initial conditions:** Small changes in starting values lead to vastly different trajectories
2. **Ergodicity:** The system's trajectory eventually visits all regions of state space
3. **Mixing property:** Points that start close together become distributed throughout the state space
4. **Strange attractors:** The system evolves toward complex geometric structures

Visual Representation

When visualized, the Tinkerbell map creates beautiful and complex patterns known as attractors. The shape and structure of these attractors depend on the parameter values and initial conditions.

 Tinkerbell Attractor

Classic Tinkerbell attractor with parameters $a=0.9$, $b=-0.6013$, $c=2.0$, $d=0.5$

3. Implementation Details

Code Structure

Our implementation of the Tinkerbell map consists of three main components:

1. **TinkerbellParams struct:** Holds the system parameters (a , b , c , d) and current state (x , y)
2. **TinkerCoeffs struct:** Contains polynomial mixing coefficients for enhanced entropy
3. **Tinkerbell class:** Implements the core functionality

Key Features

Parameter Initialization

The constructor derives parameters from a 16-byte cryptographic key:

```
Tinkerbell::Tinkerbell(const uint8_t chaosKey[16]) {  
    // Map key bytes to parameter ranges  
    p_.a = mapRange16(v0, -1.2f, -0.3f);    // a  
    p_.b = mapRange16(v1, -1.0f, -0.2f);    // b  
    p_.c = mapRange16(v2, 1.5f, 2.5f);      // c  
    p_.d = mapRange16(v3, 0.3f, 1.3f);      // d  
  
    // Initialize state variables  
    p_.x = fracFrom16b(chaosKey[8], chaosKey[9]);  
    p_.y = fracFrom16b(chaosKey[10], chaosKey[11]);  
  
    // Derive polynomial coefficients  
    // ...  
  
    // Burn-in period: 128 iterations  
    for (int i = 0; i < 128; ++i) (void)nextByte();  
}
```

Map Iteration

The core of the implementation is the `nextBit()` function, which iterates the Tinkerbell map:

```
inline uint8_t Tinkerbell::nextBit() {  
    // Iterate Tinkerbell (float)  
    float xn = p_.x, yn = p_.y;  
    float x1 = xn*xn - yn*yn + p_.a * xn + p_.b * yn;  
    float y1 = 2.0f * xn * yn + p_.c * xn + p_.d * yn;  
    p_.x = x1; p_.y = y1;  
  
    // Polynomial mixing and whitening  
    // ...  
}
```

```
return (uint8_t)(mix & 1u);  
}
```

Enhanced Features

Our implementation includes several enhancements for cryptographic strength:

1. **Polynomial Mixing:** Adds higher-order nonlinearities through a polynomial function:

$$P(x, y) = \alpha \cdot x + \beta \cdot y + \gamma \cdot x^2 + \delta \cdot y^2 + \varepsilon \cdot x \cdot y + \zeta \cdot x^3 + \eta \cdot y^3$$

2. **Whitening Process:** Removes statistical biases through xorshift-based techniques
 3. **Burn-in Period:** 128 iterations to reach the chaotic regime before generating output
-

4. Cryptographic Applications

Keystream Generation

The primary application of our Tinkerbelle implementation is generating a pseudo-random keystream for cryptographic purposes:

```
void Tinkerbelle::generate(uint8_t* out, size_t bytes) {  
    for (size_t i = 0; i < bytes; ++i) out[i] = nextByte();  
}
```

This keystream can be used in various cryptographic scenarios:

1. **Stream Cipher:** XOR the keystream with plaintext to produce ciphertext

```
void Tinkerbelle::xorBitwise(uint8_t* data, size_t len) {  
    for (size_t i = 0; i < len; ++i) data[i] ^= nextByte();  
}
```

2. **Key Derivation:** Generate cryptographic keys from a master key
3. **Random Number Generation:** Produce random numbers for various applications

Security Properties

Our implementation provides several security advantages:

1. **Large Key Space:** 16-byte key (128 bits) maps to multiple float parameters
 2. **Nonlinear Dynamics:** Chaotic behavior ensures unpredictability
 3. **Parameter Sensitivity:** Small key changes produce vastly different outputs
 4. **Polynomial Mixing:** Higher-degree nonlinearities resist cryptanalysis
 5. **Robust Whitening:** Removes statistical biases in the output
 6. **Implementation Security:** MCU-friendly operations with burn-in period
-

5. Performance and Statistical Analysis

Output Characteristics

The Tinkerbell map, when properly implemented with whitening, produces output with good statistical properties:

 Cryptographic Visualization

Visualization of the Tinkerbell map's cryptographic properties

Key statistical properties include:

- Uniform byte distribution after whitening
- Low autocorrelation between output bytes
- Passes standard randomness tests

Performance Considerations

The implementation is designed to be efficient and suitable for resource-constrained environments:

1. Computational Efficiency:

- Uses basic arithmetic and bitwise operations
- No expensive operations like division or transcendental functions
- 8 iterations per output byte balances security and performance

2. Memory Footprint:

- Small state size (parameters + current position)
- Minimal temporary storage requirements

3. Implementation Flexibility:

- Can be adapted for systems without floating-point units
 - Suitable for microcontrollers and embedded systems
-

6. Comparison with Other Chaotic Maps

Advantages of Tinkerbell Map

Compared to other chaotic maps, the Tinkerbell map offers several advantages:

1. **Two-dimensional system:** More complex dynamics than 1D maps like the logistic map
2. **Rich parameter space:** Multiple parameters allow for diverse behaviors
3. **Visually distinctive:** Creates beautiful and complex attractors
4. **Well-studied:** Mathematical properties are well-understood

Other Notable Chaotic Maps

- **Logistic Map:** One-dimensional, simpler but less secure
 - **Hénon Map:** Two-dimensional, similar complexity to Tinkerbell
 - **Arnold's Cat Map:** Used primarily for image encryption
 - **Lorenz System:** Continuous-time system, more complex to implement
-

7. Conclusion and Future Directions

Summary

The Tinkerbell chaotic map provides an elegant and effective foundation for cryptographic applications:

- Simple mathematical formulation but complex behavior
- Enhanced security through polynomial mixing and whitening
- Efficient implementation suitable for resource-constrained environments
- Visually fascinating attractors that illustrate chaos theory principles

Future Directions

Potential areas for further research and development include:

1. **Hardware optimization:** FPGA or ASIC implementations for high-speed applications

2. **Parameter optimization:** Finding optimal parameter ranges for specific security requirements
3. **Hybrid systems:** Combining Tinkerbell with other chaotic maps for enhanced security
4. **Formal security analysis:** Rigorous cryptanalysis against modern attack techniques

Final Thoughts

The Tinkerbell map demonstrates how mathematical beauty and practical security can coexist in cryptographic design. Its combination of chaotic dynamics, implementation efficiency, and security features makes it a valuable tool in the cryptographer's toolkit.

References

1. Goldsztejn, A., Hayes, W., & Collins, P. (2011). Tinkerbell is Chaotic. *SIAM Journal on Applied Dynamical Systems*, 10(4), 1480-1501.
2. Alligood, K.T., Sauer, T.D., & Yorke, J.A. (1996). *Chaos: An Introduction to Dynamical Systems*. Berlin: Springer-Verlag.
3. Alvarez, G., & Li, S. (2006). Some basic cryptographic requirements for chaos-based cryptosystems. *International Journal of Bifurcation and Chaos*, 16(08), 2129-2151.
4. Kocarev, L. (2001). Chaos-based cryptography: a brief overview. *IEEE Circuits and Systems Magazine*, 1(3), 6-21.